



Course Name	Microcontroller Based System Design
-------------	-------------------------------------

Course Code	EEE 4705
-------------	----------

Name	Student ID
Munshaat Atiq Nuzaba	210021210
Zahir Sadik	210021228
Abrar Mubasshir Rahman	210021230
Faria Ahmed Suprova	210021236
Fattaur Rahim	210021240
Musa Shuaibu	210021264

Introduction

Problem Statement

This project focuses on designing a standalone embedded Tic-Tac-Toe gaming device using the AT89C51 (8051) microcontroller, programmed entirely in Assembly Language. The system must provide a simple and intuitive retro-style user experience while operating under constraints of cost, power, memory, and I/O resources.

The device supports both Player-vs-Player (PvP) and Player-vs-AI (PvAI) modes and is validated through simulation. The challenge lies in efficiently implementing game logic, user interaction, and AI within the limited hardware capabilities of the microcontroller.

Design Objectives

- Design a low-cost embedded system using AT89C51
- Implement a 3×3 Tic-Tac-Toe game in Assembly
- Support PvP and PvAI modes
- Provide clear visualization using an LCD
- Enable user input via a keypad
- Implement rule-based AI
- Optimize I/O usage and component count
- Analyze power consumption and cost feasibility

Engineering Constraints

Cost

The system must remain low-cost; hence simple components like a character LCD and matrix keypad are used instead of advanced interfaces.

Power

Designed for low-power operation suitable for battery use, ensuring minimal current consumption.

Memory

Limited internal RAM requires efficient data usage. The game board is stored in nine memory locations (CELL1–CELL9) along with minimal control variables.

I/O

Limited pins are optimized as:

- Port 1 → Keypad
- Port 2 → LCD data (4-bit mode)
- Port 3 → LCD control

Usability

The system ensures:

- Easy mode selection
- Clear turn indication
- Invalid move detection
- Proper win/draw display

Project Scope

The project includes:

- Input/output selection and justification
- Hardware design with AT89C51
- Assembly-based game and AI implementation
- Simulation and validation
- Power and cost analysis
- Technical documentation

System Architecture:

The complete system is composed of the following major subsystems:

i) Input Unit- 4x4 Matrix keypad

This unit serves as the primary user input interface, allowing the player to select the game mode, choose board positions during gameplay and reset the system or return to the main menu. When a key is pressed, the microcontroller detects it through a row-column scanning process.

ii) Processing Unit - AT89C51 Microcontroller

All the decision making and coordination of the game occur inside this box, making it the heart of the system. Its key responsibilities include:

- Initializing the system at startup
- Reading keypad inputs
- Storing the board state in internal RAM
- Validating moves
- Switching turns in between players
- Checking for win and draw conditions
- Updating the output display

iii) Output Unit - 20x4 LCD Display

This is the main visual output device of the system. It displays the 3x3 Tic-Tac-Toe board, the current turn (X or O), the selected mode (PvP or AI mode), invalid move messages and the final outcome.

iv) Status Indicator Unit - LEDs

The two LEDs here work as auxiliary visual indicators. They can be used to indicate system status, active conditions or provide extra visual feedback during operation alongside the LCD display. Their inclusion improves the visibility of the device and adds a simple hardware-level output feature with minimal cost and complexity.

v) Clock Generation Circuit

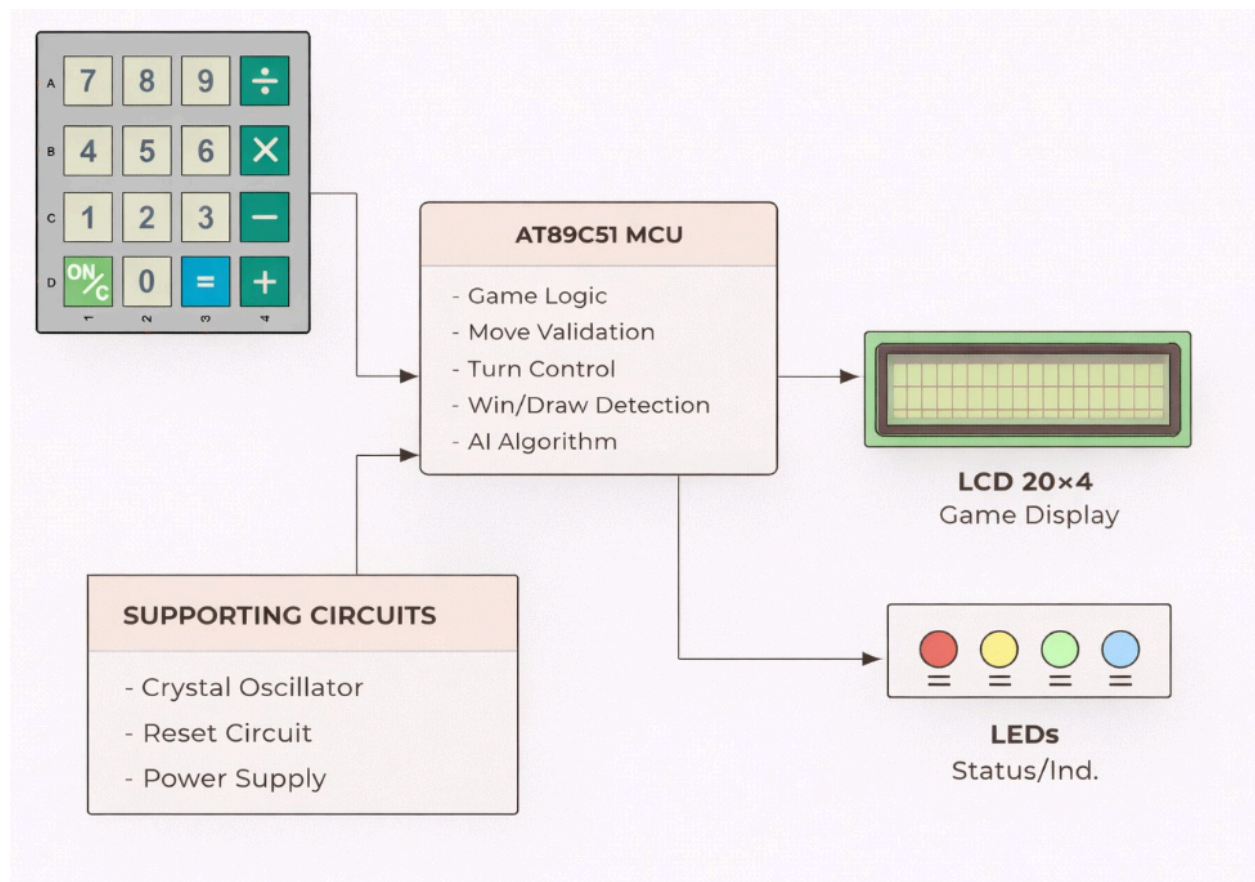
The crystal oscillator and its capacitors provide the timing signal required by the microcontroller. This block ensures stable and reliable system operation.

vi) Reset Circuit

This circuit ensures that the system starts from a known initial condition whenever the device is powered on or when the reset button is pressed. It forces the microcontroller to restart program execution from the beginning.

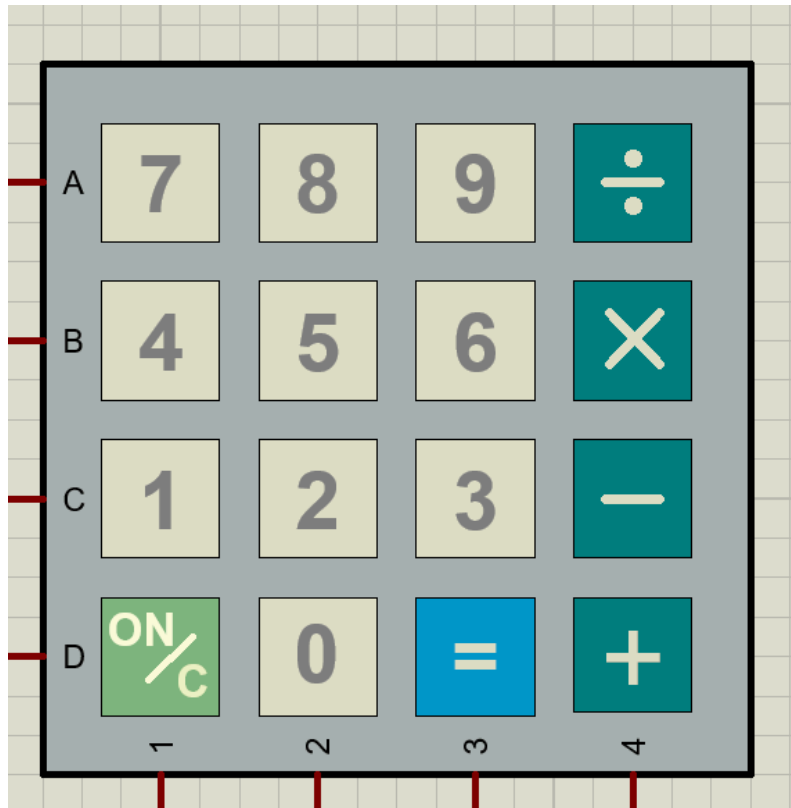
When the user presses a key on the keypad, the keypad sends the input signal to the microcontroller. The microcontroller processes the input according to the game logic, the LCD shows a main menu where the player can select PvP or PvAI mode. After selection, the board is cleared, turn is initialized to X and gameplay begins. A keypad input is read and decoded into a board cell address. If the move is valid, the corresponding memory cell is updated. The system then checks for win or draw. In case of PvAI mode activation and it is O's turn, the AI routine executes. The LCD refreshes the board and status line after each event. On game completion, the system waits for reset to return to the menu.

Block Diagram



Input and Output Unit Selection:

The selected input device is a 4x4 matrix keypad.



It is supported in the source code through a keypad scan routine that drives rows and reads columns using Port 1. This keypad served multiple purposes including:

- Keys corresponding to board positions 1 to 9
- Menu selection for PvP
- Menu selection for PvAI
- Reset to menu

In the main menu, logic checks keypad values for selecting game mode and reset. Here one key selects PvP, one key selects PvAI and one key resets or returns to the main menu.

The keypad is a suitable choice because it provides multiple inputs with only 8 microcontroller pins, it is cheaper than using 16 individual push buttons, it supports both numeric board selection and menu commands and it is familiar, easy to use.

Comparative Analysis with Alternatives:

a) Individual Push Buttons

A possible alternative would be to use 9 separate buttons for the board and additional for mode/reset. However, that would require more I/O pins, more PCB wirings and physical size and cost would increase.

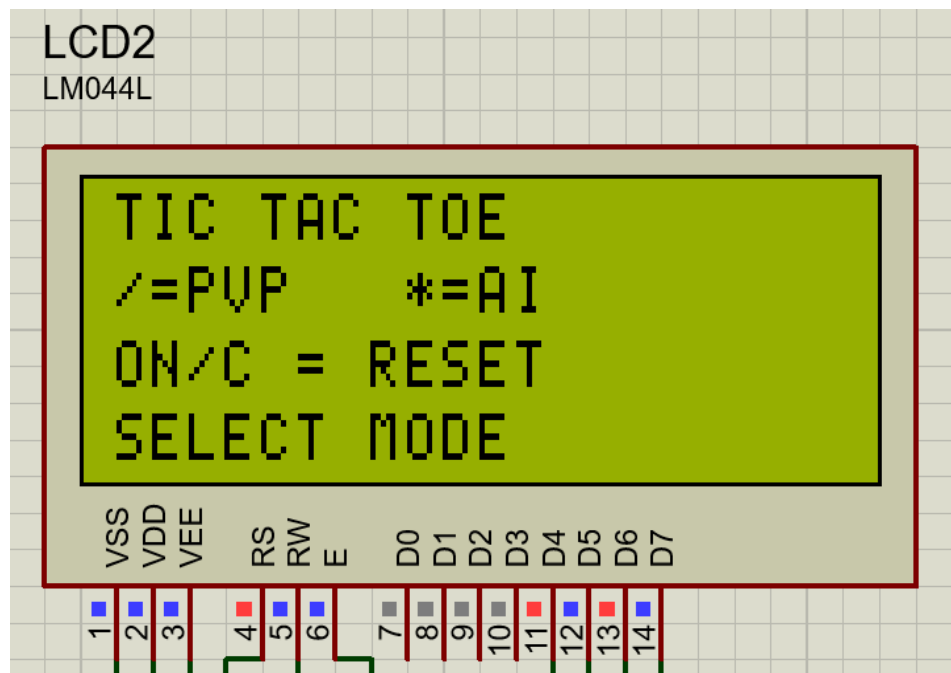
b) Capacitive Touch Keys

Using them would improve the performance but they are more expensive, would require more complex sensing circuitry and would increase implementation complexity beyond the goals of a low-cost 8051 design.

c) Joystick

A joystick could move a cursor and selecting a key could confirm a move but in that case, user interaction becomes slower, extra software for cursor handling is required and it is less intuitive than direct cell entry.

The output unit is a 20x4 character LCD module (LM044L). It is operated in 4 bit mode. LCD is particularly chosen for this application because it can display three game rows, turn status, menu text, invalid move message and the final result (win/lose/draw).



Comparative Analysis with Alternatives:

a) Single LEDs

They are only useful for binary indication, and can not display the entire game board.

b) 7 segment displays

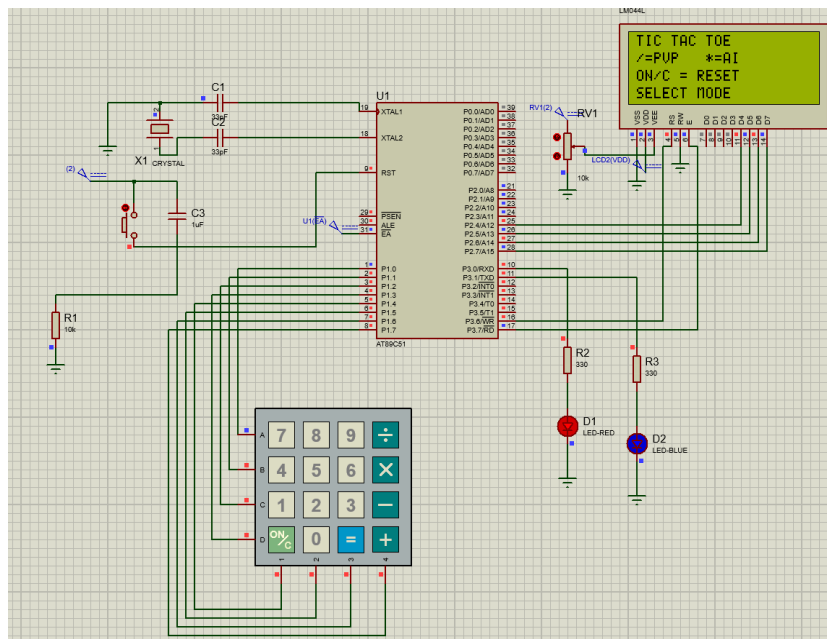
It would be good for numeric display but unsuitable for showing 3x3 game board and status messages.

c) Graphical LCD

It would provide better aesthetics but it is more expensive, more memory intensive, has more complex driver software and unnecessary for a low-cost retro target.

The LCD improves usability by making the system self-explanatory. The board labels, player turn, mode information and final results all are directly visible, which is much better than minimal output solutions and satisfies the usability requirement well.

Hardware Design:



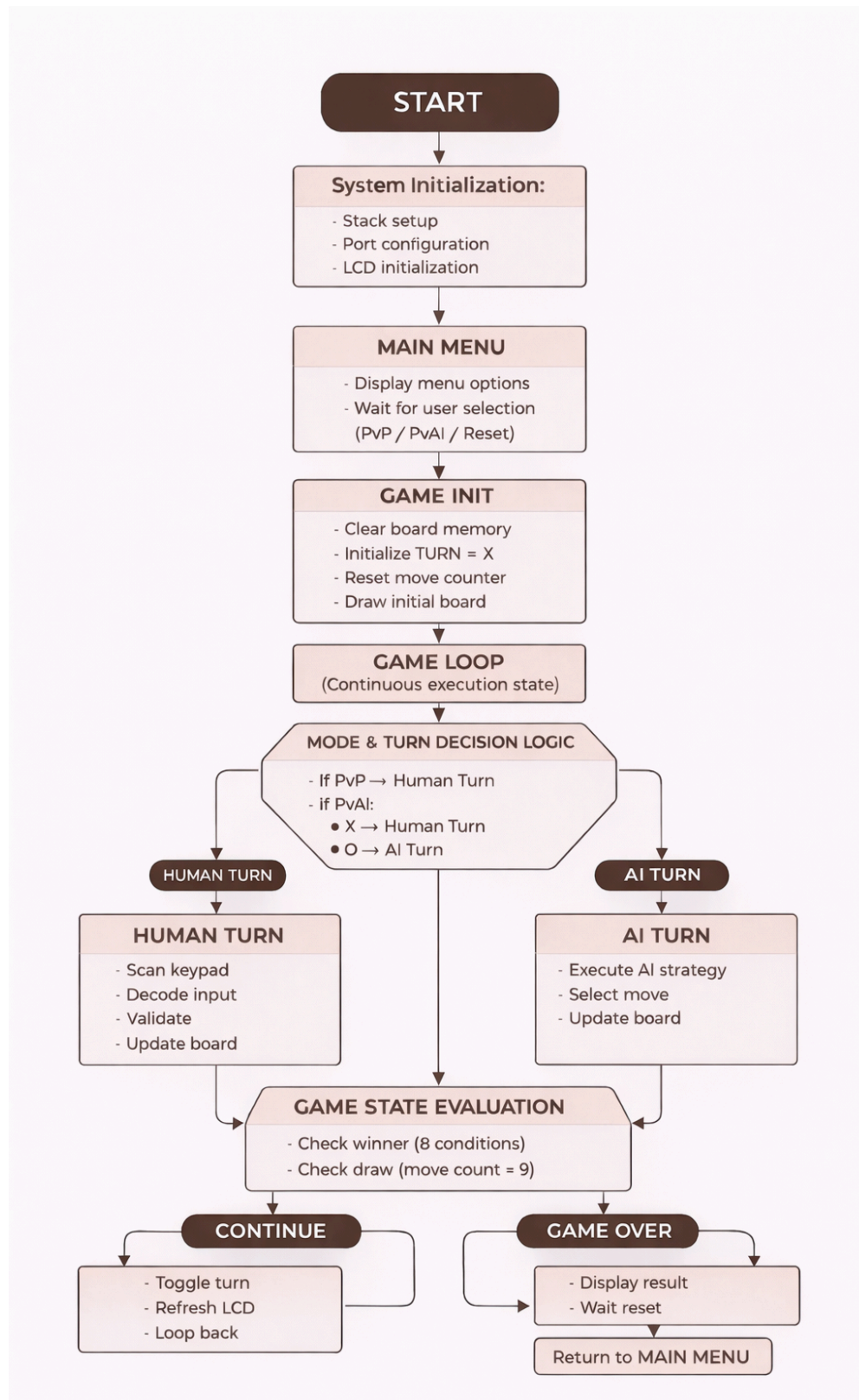
Software Design

The software is written entirely in 8051 assembly language. It follows a modular and state-driven design. The program is structured around a continuous loop that transitions between different operational states such as:

- Main Menu
- Game Initialization
- Player Turns (Human / AI)
- Game Over

Each functionality (input handling, display update, game logic, AI decision-making) is implemented as separate subroutines.

Flowchart:



Program Modules:

Module	Subroutine / Label	Function Description
System Initialization	START, LCD_INIT	Initialize system, ports, and LCD
Main Menu	MAIN_MENU, SHOW_MENU	Display menu and get mode input
Game Initialization	GAME_INIT, CLEAR_BOARD	Reset board, turn, and counters
Game Loop	GAME_LOOP	Control overall game flow
Input Handling	GET_KEY_BLOCK, SCAN_KEY	Read and debounce keypad input
Key Mapping	KEY_TO_ADDR	Map key to board position
Move Validation	CHK_GAME_KEY, PLACE_HUMAN	Validate and place player move
Turn Management	TOGGLE_TURN	Switch between X and O
Board Display	DRAW_BOARD, GET_CELL_CHAR	Update LCD with board state

Win Detection	CHECK_WINNER	Check all win conditions
Draw Detection	CHECK_DRAW	Check if game is draw
AI Control	AI_MOVE	Manage AI decisions
AI Win	TRY_WIN_O	Try to win
AI Block	TRY_BLOCK_X	Block opponent
AI Fork	TRY_FORK_O	Create multiple threats
AI Block Fork	TRY_BLOCK_FORK_X	Prevent opponent forks
Threat Eval	COUNT_THREATS	Count possible threats
Strategy Moves	TRY_OPP_CORNER, TRY_ANY_CORNER, TRY_ANY_SIDE	Select strategic positions
Line Eval	TRY_TWO_ANY	Find 2-in-a-row opportunities
Move Execution	PLACE_O_AT_ADDR	Place AI move
Game Over	GAME_OVER	Show result and wait reset
Error Handling	SHOW_INVALID	Handle invalid moves

LCD Control	LCD_CMD, LCD_DATA, LCD_CLEAR, LCD_PRINT_STRING	Control LCD operations
Timing	DELAY_TINY, DELAY_SHORT, DELAY_BIG, DELAY_DEB	Provide delays

Game Logic:

Input Handling

The keypad input is handled using the **SCAN_KEY** routine, which continuously scans for a key press. The **GET_KEY_BLOCK** routine ensures reliable input by performing debouncing, detecting when a key is pressed, and confirming when it is released before accepting it as valid input.

Move Processing

Once a key is detected, it is converted into the corresponding board position using **KEY_TO_ADDR**. The system then checks whether the selected cell is empty. If the cell is not empty, the move is considered invalid. If the cell is empty, the current player's mark (X or O) is stored in that position, and the move counter is incremented.

Turn Switching

Turn switching is managed by the **TOGGLE_TURN** routine. If the current turn is X, it switches to O; otherwise, it switches back to X after each valid move.

Win Detection

The **CHECK_WINNER** routine evaluates all possible winning combinations on the board, including three rows, three columns, and two diagonals. It checks whether three cells in any of these combinations contain the same non-zero value. If such a condition is met, the system declares a winner by setting the **WINNER** variable to either XMARK or OMARK.

Draw Detection

The `CHECK_DRAW` routine determines whether the game ends in a draw. It checks if the move counter has reached 9 and no winner has been declared. If both conditions are satisfied, the system sets the `WINNER` variable to indicate a draw (03H).

AI Algorithm Design

The AI in this Tic-Tac-Toe system is designed using a rule-based approach that follows a fixed priority sequence. Instead of using complex algorithms, it makes decisions by checking for winning opportunities, blocking the opponent, and selecting strategic positions. This ensures efficient and reliable performance suitable for an embedded system.

Sequence of Operation

The AI move is controlled by the `AI_MOVE` routine, which follows a step-by-step priority sequence:

1. The AI first attempts to win using `TRY_WIN_O`. It checks all rows, columns, and diagonals to find a line with two O marks and one empty cell. If found, it places O in the empty cell to win the game.
2. If no winning move is available, the AI tries to block the opponent using `TRY_BLOCK_X`. It looks for any line where the opponent (X) has two marks and one empty cell, and places O there to prevent a loss.
3. If neither winning nor blocking is possible, the AI attempts to create a fork using `TRY_FORK_O`. This means placing a mark in a position that creates multiple future winning opportunities simultaneously.
4. If the opponent has a potential fork opportunity, the AI uses `TRY_BLOCK_FORK_X` to block it by occupying a critical position.
5. If no immediate threats or opportunities exist, the AI selects positions based on strategy. It may choose the opposite corner of the opponent (`TRY_OPP_CORNER`), any available corner (`TRY_ANY_CORNER`), or a side position (`TRY_ANY_SIDE`) to maintain board advantage.
6. During decision-making, supporting routines like `COUNT_THREATS` and `TRY_TWO_ANY` help evaluate possible lines with two marks and one empty cell.
7. Finally, once the best move is determined, the AI places its mark using `PLACE_O_AT_ADDR` and updates the move counter.

Power Consumption and Cost Analysis

Bill of Materials (BOM)

Component	Quantity	Unit Cost (Approx.)	Total Cost
AT89C51 Microcontroller	1	250 BDT	250 BDT
20×4 LCD Module (LM044L or equivalent)	1	550 BDT	550 BDT
4×4 Matrix Keypad	1	80 BDT	80 BDT
Crystal Oscillator	1	20 BDT	20 BDT
Small Capacitors	2	5 BDT	10 BDT
Electrolytic Capacitor	1	10 BDT	10 BDT
10 kΩ Resistors	2	2 BDT	4 BDT
330 Ω Resistors	2	2 BDT	4 BDT
LEDs	2	5 BDT	10 BDT
Contrast Potentiometer	1	55 BDT	55 BDT
Push Button	1	10 BDT	10 BDT
Misc. wiring / PCB / connector allowance	1 set	100 BDT	100 BDT

Estimated total cost 1103 BDT, which confirms that the design is reasonably low cost.

Power Consumption Estimate

Practical estimation of current of different components are:

Component	Approx. Current
AT89C51	15–25 mA
20×4 LCD logic + backlight	15–30 mA
2 LEDs	8–12 mA
Keypad	negligible
Misc. losses	2–5 mA

Total current 45 mA to 70 mA

For calculation, a nominal value of 55 mA is used.

Here

$$\text{Operating Time (hours)} = \frac{\text{Battery Capacity (mAh)}}{\text{Load Current (mA)}}$$

For a 1000 mAh battery, operating time is $1000/55 = 18.18$ hours.

For a 2000 mAh battery, operating time is $2000/55 = 36.36$ hours.

Thus, even a modest battery can run the system for a long period, making the design practical for handheld operation. The design keeps power moderate because the keypad consumes almost no static power, the LCD is more efficient than a dense LED matrix and the 8051 is relatively light-weight.

Individual Contribution

1) Munshaat Atiq Nuzaba, ID 210021210

- Designed and implemented the rule-based AI algorithm for PvAI mode using prioritized decision-making logic
- Developed advanced AI features including win detection, opponent blocking, fork creation, fork prevention, and positional strategies

- Tested AI performance under different gameplay scenarios to ensure correct and intelligent responses
- Conducted overall system testing and validation, identifying bugs, verifying edge cases, and ensuring reliable system performance

2) Zahir Sadik, ID 210021228

- Selected and justified the 4×4 keypad and 20×4 LCD, supported by comparative analysis based on cost, I/O usage, readability, and complexity
- Designed LCD layout including board display, turn indication, mode info, and result/error messages
- Defined full keypad mapping for board positions, mode selection, and control functions
- Optimized I/O usage and provided complete interface specifications ensuring consistency between hardware and software implementation

3) Abrar Mubasshir Rahman, ID 210021230

- Designed and implemented the complete hardware schematic in Proteus, including AT89C51 microcontroller, keypad, LCD, LEDs, crystal oscillator, reset circuit, and supporting components
- Configured proper interfacing between all components, ensuring correct pin connections, voltage levels, and signal flow across the system
- Implemented and verified supporting circuits such as the clock generation unit and reset mechanism for stable system operation
- Performed full simulation and debugging in Proteus, validating hardware behavior, checking input/output response, and ensuring proper integration with the software

4) Faria Ahmed Suprova, ID 210021236

- Developed the complete program in 8051 Assembly Language, including initialization, menu system, and main game loop
- Implemented core game logic such as keypad input handling, board update mechanism, turn switching, and win/draw condition checking
- Designed and integrated LCD interface routines for displaying the board, status messages, and results in real time
- Debugged and optimized the code to ensure correct functionality, efficient memory usage, and smooth interaction between hardware and software

5) Fattaur Rahim , ID 210021240

- Designed the high-level system architecture and developed the complete block diagram showing input, processing, output, and supporting units
- Defined data flow, signal paths, and control relationships between keypad, AT89C51, LCD, and LEDs
- Created pin allocation plan and analyzed microcontroller constraints to guide system design decisions
- Designed memory organization for board state, turn tracking, and flags, and prepared system-level documentation for the report

6) Musa Shuaibu, ID 210021264

- Coordinated between hardware and software teams and verified complete system operation in Proteus, including all scenarios and edge cases
- Defined core game logic such as turn switching, win/draw conditions, and end-game behavior

Conclusion:

This project successfully demonstrates the design of a complete microcontroller-based Tic-Tac-Toe gaming system using the AT89C51 and 8051 assembly language. The final design supports both the PvP and PvAI mode. The input unit uses a 4x4 keypad and the output unit uses a 20x4 LCD display. From an engineering perspective, the project meets the intended goals of low cost, acceptable power consumption, compact I/O utilization and clear user interaction. Despite its success, the current system has several limitations:

- It uses a character LCD rather than a graphical display.
- It has no sound/buzzer feedback.
- There is no persistent score storage.

Future work may include:

- Replacing the character LCD with a graphical OLED or TFT
- Adding sound effects using buzzer
- Reducing power using idle mode or sleep management

Appendix:

```

1 ;=====
2 ; Tic-Tac-Toe for AT89C51
3 ; LCD 20x4 in 4-bit mode | Keypad 4x4
4 ;
5 ; CHANGES FROM ORIGINAL:
6 ;   BUG FIX 1 - P3 init: was #00H, corrected to #0FFH
7 ;   BUG FIX 2 - Status line: DRAW_BOARD now called AFTER
8 ;               TOGGLE_TURN so correct player's turn shown
9 ;   BUG FIX 3 - Double draw: removed stray DRAW_BOARD
10 ;              from inside AI_DONE; board only drawn once
11 ;              per event now
12 ;   NEW AI    - 7-priority enhanced rule-based AI:
13 ;               1. Win (complete own line)
14 ;               2. Block (stop opponent win)
15 ;               3. Fork Attack (create 2 threats at once)
16 ;               4. Block Fork (prevent opponent fork)
17 ;               5. Center
18 ;               6. Opposite Corner (counter corner play)
19 ;               7. Any Corner
20 ;               8. Any Side
21 ;=====
22
23         ORG     0000H
24         AJMP    START
25
26 ;=====
27 ; CONSTANTS
28 ;=====
29 XMARK     EQU     01H
30 OMARK     EQU     02H
31
32 MODE_PVP   EQU     00H
33 MODE_AI    EQU     01H
34
35 ;=====
36 ; RAM MAP
37 ;=====
38 CELL1     EQU     30H
39 CELL2     EQU     31H
40 CELL3     EQU     32H
41 CELL4     EQU     33H
42 CELL5     EQU     34H
43 CELL6     EQU     35H
44 CELL7     EQU     36H
45 CELL8     EQU     37H
46 CELL9     EQU     38H
47
48 MODEVAR    EQU     39H
49 TUDM      EQU     3AH

```

```

49  TURN      EQU      3AH
50  WINNER    EQU      3BH
51  KEYVAL    EQU      3CH
52  MOVECNT   EQU      3DH
53  FLAGS     EQU      3EH
54  TEMP      EQU      3FH
55
56  ADR1      EQU      40H
57  ADR2      EQU      41H
58  ADR3      EQU      42H
59  VAL1      EQU      43H
60  VAL2      EQU      44H
61  VAL3      EQU      45H
62  TARGET    EQU      46H
63
64  ; --- New variables for enhanced AI fork detection ---
65  FORK_CELL  EQU      47H      ; cell RAM address under fork test (30H..38H)
66  THREAT_CNT EQU      48H      ; threat count returned by COUNT_THREATS
67
68  ;=====
69  ; START
70  ;=====
71  START:
72      MOV     SP,#70H
73
74      MOV     P0,#0FFH
75      MOV     P1,#0FFH
76      MOV     P2,#00H
77      MOV     P3,#0FFH      ; BUG FIX 1: was #00H (pulled INT/UART pins low)
78
79      ACALL   LCD_INIT
80
81  ;=====
82  ; MAIN MENU
83  ;=====
84  MAIN_MENU:
85      ACALL   CLEAR_BOARD
86      MOV     MODEVAR,#00H
87      MOV     TURN,#XMARK
88      MOV     WINNER,#00H

```

Line: 1264 Column: 11 Insert .asm MCU 8051

```

89         MOV     MOVECNT, #00H
90         ACALL   SHOW_MENU
91
92 WAIT_MODE:
93         ACALL   GET_KEY_BLOCK
94         MOV     KEYVAL, A
95
96         CJNE    A, #0AH, CHK_AI_MODE      ; Row-A col-4 => PvP
97         MOV     MODEVAR, #MODE_PVP
98         LJMP    GAME_INIT
99
100 CHK_AI_MODE:
101         CJNE    A, #0BH, CHK_MENU_RESET   ; Row-B col-4 => PvAI
102         MOV     MODEVAR, #MODE_AI
103         LJMP    GAME_INIT
104
105 CHK_MENU_RESET:
106         CJNE    A, #0CH, WAIT_MODE        ; ON/C key => reset
107         LJMP    MAIN_MENU
108
109 ;=====
110 ; GAME INIT
111 ;=====
112 GAME_INIT:
113         ACALL   CLEAR_BOARD
114         MOV     TURN, #XMARK
115         MOV     WINNER, #00H
116         MOV     MOVECNT, #00H
117         ACALL   DRAW_BOARD
118
119 ;=====
120 ; GAME LOOP
121 ;=====
122 GAME_LOOP:
123         MOV     A, MODEVAR
124         JZ      HUMAN_TURN                ; PVP => always human
125
126         MOV     A, TURN
127         CJNE    A, #OMARK, HUMAN_TURN     ; AI mode but X turn => human
128         ACALL   AI_MOVE
129         ACALL   CHECK_WINNER
130         MOV     A, WINNER
131         JNZ     GAME_OVER
132         ACALL   CHECK_DRAW
133         MOV     A, WINNER

```

```

134             JNZ     GAME_OVER
135             ACALL    TOGGLE_TURN
136             ACALL    DRAW_BOARD           ; BUG FIX 2: draw AFTER toggle
137             LJMP     GAME_LOOP           ;           => shows "X TURN" correctly
138
139             ;=====
140             ; HUMAN TURN
141             ;=====
142 HUMAN_TURN:
143 WAIT_HUMAN_KEY:
144             ACALL    GET_KEY_BLOCK
145             MOV      KEYVAL,A
146
147             CJNE     A,#0CH,CHK_GAME_KEY
148             LJMP     MAIN_MENU
149
150 CHK_GAME_KEY:
151             ACALL    KEY_TO_ADDR
152             JNC      WAIT_HUMAN_KEY       ; invalid key code, ignore
153
154             MOV      A,@R0
155             JZ        PLACE_HUMAN
156             ACALL    SHOW_INVALID         ; cell occupied
157             LJMP     WAIT_HUMAN_KEY
158
159 PLACE_HUMAN:
160             MOV      A,TURN
161             MOV      @R0,A
162             INC      MOVECNT
163             ; BUG FIX 2+3: removed early DRAW_BOARD here;
164             ; now drawn after TOGGLE_TURN so status is correct
165             LCALL    CHECK_WINNER
166             MOV      A,WINNER
167             JNZ      GAME_OVER
168             LCALL    CHECK_DRAW
169             MOV      A,WINNER
170             JNZ      GAME_OVER
171             LCALL    TOGGLE_TURN
172             ACALL    DRAW_BOARD           ; draw after toggle => correct next-turn status
173             LJMP     GAME_LOOP
174
175             ;=====
176             ; GAME OVER
177             ;=====
178 GAME_OVER:

```

```

180
181 WAIT_RESET_KEY:
182     LCALL    GET_KEY_BLOCK
183     CJNE     A,#0CH,WAIT_RESET_KEY
184     LJMP     MAIN_MENU
185
186 ;=====
187 ; CLEAR BOARD
188 ;=====
189 CLEAR_BOARD:
190     MOV      CELL1,#00H
191     MOV      CELL2,#00H
192     MOV      CELL3,#00H
193     MOV      CELL4,#00H
194     MOV      CELL5,#00H
195     MOV      CELL6,#00H
196     MOV      CELL7,#00H
197     MOV      CELL8,#00H
198     MOV      CELL9,#00H
199     RET
200
201 ;=====
202 ; TOGGLE TURN
203 ;=====
204 TOGGLE_TURN:
205     MOV      A,TURN
206     CJNE     A,#XMARK,SET_X_TURN
207     MOV      TURN,#OMARK
208     RET
209 SET_X_TURN:
210     MOV      TURN,#XMARK
211     RET
212
213 ;=====
214 ; KEY -> CELL ADDRESS
215 ; Input : A = key code (01H..09H = cells 1..9)
216 ; Output: R0 = cell RAM address, C=1 if valid
217 ;         C=0 if key code not a board cell
218 ;=====
219 KEY_TO_ADDR:
220     CJNE     A,#01H,KT2
221     MOV      R0,#CELL1
222     SETB     C
223     RET
224 KT2:      CJNE     A,#02H,KT3

```

```

224     KT2:      CJNE     A,#02H,KT3
225             MOV      R0,#CELL2
226             SETB     C
227             RET
228     KT3:      CJNE     A,#03H,KT4
229             MOV      R0,#CELL3
230             SETB     C
231             RET
232     KT4:      CJNE     A,#04H,KT5
233             MOV      R0,#CELL4
234             SETB     C
235             RET
236     KT5:      CJNE     A,#05H,KT6
237             MOV      R0,#CELL5
238             SETB     C
239             RET
240     KT6:      CJNE     A,#06H,KT7
241             MOV      R0,#CELL6
242             SETB     C
243             RET
244     KT7:      CJNE     A,#07H,KT8
245             MOV      R0,#CELL7
246             SETB     C
247             RET
248     KT8:      CJNE     A,#08H,KT9
249             MOV      R0,#CELL8
250             SETB     C
251             RET
252     KT9:      CJNE     A,#09H,KT_BAD
253             MOV      R0,#CELL9
254             SETB     C
255             RET
256     KT_BAD:
257             CLR      C
258             RET
259
260     ;=====
261     ; DRAW BOARD ON LCD
262     ; Board layout on screen (matches numpad):
263     ;   LCD line 1:  CELL1 | CELL2 | CELL3   (keys 7 8 9)
264     ;   LCD line 2:  CELL4 | CELL5 | CELL6   (keys 4 5 6)
265     ;   LCD line 3:  CELL7 | CELL8 | CELL9   (keys 1 2 3)
266     ;   LCD line 4:  status / result message
267     ;=====
268     DRAW_BOARD:
269             LCALL     LCD_CLEAR

```



```

270
271
272 ; --- LCD Line 1 (DDRAM 0x00, cmd 0x80) ---
273 MOV     A, #080H
274 LCALL  LCD_CMD
275 MOV     A, #01H
276 LCALL  GET_CELL_CHAR
277 LCALL  LCD_DATA
278 MOV     A, #'|'
279 LCALL  LCD_DATA
280 MOV     A, #02H
281 LCALL  GET_CELL_CHAR
282 LCALL  LCD_DATA
283 MOV     A, #'|'
284 LCALL  LCD_DATA
285 MOV     A, #03H
286 LCALL  GET_CELL_CHAR
287 LCALL  LCD_DATA
288
289 ; --- LCD Line 2 (DDRAM 0x40, cmd 0xC0) ---
290 MOV     A, #0C0H
291 LCALL  LCD_CMD
292 MOV     A, #04H
293 LCALL  GET_CELL_CHAR
294 LCALL  LCD_DATA
295 MOV     A, #'|'
296 LCALL  LCD_DATA
297 MOV     A, #05H
298 LCALL  GET_CELL_CHAR
299 LCALL  LCD_DATA
300 MOV     A, #'|'
301 LCALL  LCD_DATA
302 MOV     A, #06H
303 LCALL  GET_CELL_CHAR
304 LCALL  LCD_DATA
305
306 ; --- LCD Line 3 (DDRAM 0x14, cmd 0x94) ---
307 MOV     A, #094H
308 LCALL  LCD_CMD
309 MOV     A, #07H
310 LCALL  GET_CELL_CHAR
311 LCALL  LCD_DATA
312 MOV     A, #'|'
313 LCALL  LCD_DATA
314 MOV     A, #08H
315 LCALL  GET_CELL_CHAR
316 LCALL  LCD_DATA

```

```

316         MOV     A, #'|'
317         LCALL    LCD_DATA
318         MOV     A, #09H
319         LCALL    GET_CELL_CHAR
320         LCALL    LCD_DATA
321
322         ; --- LCD Line 4 (DDRAM 0x54, cmd 0xD4) ---
323         MOV     A, #0D4H
324         LCALL    LCD_CMD
325
326         MOV     A, WINNER
327         JZ      DRAW_STATUS      ; no winner yet => show turn
328
329         CJNE    A, #XMARK, CHK_OWIN
330         MOV     DPTR, #STR_XWIN
331         LCALL    LCD_PRINT_STRING
332         RET
333
334     CHK_OWIN:
335         CJNE    A, #OMARK, SHOW_DRAW_STR
336         MOV     DPTR, #STR_OWIN
337         LCALL    LCD_PRINT_STRING
338         RET
339
340     SHOW_DRAW_STR:
341         MOV     DPTR, #STR_DRAW
342         LCALL    LCD_PRINT_STRING
343         RET
344
345     DRAW_STATUS:
346         MOV     A, MODEVAR
347         JZ      STATUS_PVP
348
349         MOV     A, TURN
350         CJNE    A, #XMARK, STATUS_AI_O
351         MOV     DPTR, #STR_AIX
352         LCALL    LCD_PRINT_STRING
353         RET
354
355     STATUS_AI_O:
356         MOV     DPTR, #STR_AIO
357         LCALL    LCD_PRINT_STRING
358         RET
359
360     STATUS_PVP:
361         MOV     A, TURN

```

Line: 1264	Column: 11	Insert	.asm	MCU 8051
------------	------------	--------	------	----------

```

361         CJNE     A,#XMARK,STATUS_PVP_O
362         MOV      DPTR,#STR_PVPX
363         LCALL    LCD_PRINT_STRING
364         RET
365 STATUS_PVP_O:
366         MOV      DPTR,#STR_PVPO
367         LCALL    LCD_PRINT_STRING
368         RET
369
370 ;=====
371 ; GET DISPLAY CHARACTER FOR CELL INDEX 1..9
372 ; If cell is empty => show keypad label from LABEL_TABLE
373 ; If cell = XMARK => 'X'
374 ; If cell = OMARK => 'O'
375 ;=====
376 GET_CELL_CHAR:
377         MOV      TEMP,A
378         ADD      A,#2FH          ; index 1 -> 30H (CELL1)
379         MOV      R0,A
380         MOV      A,@R0
381         JZ       EMPTY_CELL_CHAR
382
383         CJNE     A,#XMARK,CHK_O_CHAR
384         MOV      A,#'X'
385         RET
386 CHK_O_CHAR:
387         MOV      A,#'O'
388         RET
389
390 EMPTY_CELL_CHAR:
391         MOV      A,TEMP
392         DEC      A              ; 0-based index for LABEL_TABLE
393         MOV      DPTR,#LABEL_TABLE
394         MOVC     A,@A+DPTR
395         RET
396
397 ;=====
398 ; SHOW MENU
399 ;=====
400 SHOW_MENU:
401         LCALL    LCD_CLEAR
402
403         MOV      A,#080H
404         LCALL    LCD_CMD
405         MOV      DPTR,#STR_TITLE
406         LCALL    LCD_PRINT_STRING
407

```

Line: 1264 Column: 11 Insert .asm MCU 8051

.....  C:\Users\atiqm\Downloads\CEF | M-IDE Studio for MCS-51 : Release 0.2.5.8

```

408         MOV     A, #0C0H
409         LCALL   LCD_CMD
410         MOV     DPTR, #STR_MODE1
411         LCALL   LCD_PRINT_STRING
412
413         MOV     A, #094H
414         LCALL   LCD_CMD
415         MOV     DPTR, #STR_MODE2
416         LCALL   LCD_PRINT_STRING
417
418         MOV     A, #0D4H
419         LCALL   LCD_CMD
420         MOV     DPTR, #STR_MODE3
421         LCALL   LCD_PRINT_STRING
422         RET
423
424         ;=====
425         ; SHOW INVALID MOVE MESSAGE
426         ;=====
427 SHOW_INVALID:
428         MOV     A, #0D4H
429         LCALL   LCD_CMD
430         MOV     DPTR, #STR_INVALID
431         LCALL   LCD_PRINT_STRING
432         LCALL   DELAY_BIG
433         LCALL   DRAW_BOARD           ; restore normal board display
434         RET
435
436         ;=====
437         ; CHECK WINNER
438         ; Tests all 8 winning lines.
439         ; Sets WINNER = XMARK / OMARK if found, 00H if none.
440         ;=====
441 CHECK_WINNER:
442         MOV     WINNER, #00H
443
444         ; Row 1-2-3
445         MOV     A, CELL1
446         JZ      CW2
447         CJNE    A, CELL2, CW2
448         CJNE    A, CELL3, CW2
449         MOV     WINNER, A
450         RET
451 CW2:
452         ; Row 4-5-6

```

```

452             ; Row 4-5-6
453             MOV     A, CELL4
454             JZ       CW3
455             CJNE     A, CELL5, CW3
456             CJNE     A, CELL6, CW3
457             MOV     WINNER, A
458             RET
459 CW3:
460             ; Row 7-8-9
461             MOV     A, CELL7
462             JZ       CW4
463             CJNE     A, CELL8, CW4
464             CJNE     A, CELL9, CW4
465             MOV     WINNER, A
466             RET
467 CW4:
468             ; Col 1-4-7
469             MOV     A, CELL1
470             JZ       CW5
471             CJNE     A, CELL4, CW5
472             CJNE     A, CELL7, CW5
473             MOV     WINNER, A
474             RET
475 CW5:
476             ; Col 2-5-8
477             MOV     A, CELL2
478             JZ       CW6
479             CJNE     A, CELL5, CW6
480             CJNE     A, CELL8, CW6
481             MOV     WINNER, A
482             RET
483 CW6:
484             ; Col 3-6-9
485             MOV     A, CELL3
486             JZ       CW7
487             CJNE     A, CELL6, CW7
488             CJNE     A, CELL9, CW7
489             MOV     WINNER, A

```

Line: 1264 Column: 11 Insert .asm MCU 8051

```

489             MOV     WINNER,A
490             RET
491 CW7:
492             ; Diagonal 1-5-9
493             MOV     A,CELL1
494             JZ      CW8
495             CJNE    A,CELL5,CW8
496             CJNE    A,CELL9,CW8
497             MOV     WINNER,A
498             RET
499 CW8:
500             ; Diagonal 3-5-7
501             MOV     A,CELL3
502             JZ      CW_END
503             CJNE    A,CELL5,CW_END
504             CJNE    A,CELL7,CW_END
505             MOV     WINNER,A
506 CW_END:
507             RET
508
509 ;=====
510 ; CHECK DRAW
511 ; If WINNER still 0 and all 9 cells filled => WINNER=03H
512 ;=====
513 CHECK_DRAW:
514             MOV     A,WINNER
515             JNZ     CD_END
516             MOV     A,MOVECNT
517             CJNE    A,#09H,CD_END
518             MOV     WINNER,#03H
519 CD_END:
520             RET
521
522 ;=====
523 ; AI MOVE - Enhanced Rule-Based (plays as OMARK)
524 ;
525 ; Priority order:
526 ; 1. WIN      - if O can complete a line, do it
527 ; 2. BLOCK    - if X is about to win, block it
528 ; 3. FORK ATTACK - find a cell that creates 2 O threats
529 ; 4. BLOCK FORK - find a cell where X would get 2 threats
530 ; 5. CENTER   - take cell 5 if free
531 ; 6. OPP CORNER - if X is in a corner, take opposite corner

```

Line: 447 Column: 32 Insert .asm MCU 8051

```

532 ; 7. ANY CORNER - take any free corner
533 ; 8. ANY SIDE - take any free side
534 ;
535 ; NOTE: No DRAW_BOARD here anymore (BUG FIX 3).
536 ; Caller draws after TOGGLE_TURN.
537 ;=====
538 AI_MOVE:
539 ; --- Priority 1: Win ---
540 LCALL TRY_WIN_O
541 JC AI_PLACED
542
543 ; --- Priority 2: Block ---
544 LCALL TRY_BLOCK_X
545 JC AI_PLACED
546
547 ; --- Priority 3: Fork Attack ---
548 LCALL TRY_FORK_O
549 JC AI_PLACED
550
551 ; --- Priority 4: Block Fork ---
552 LCALL TRY_BLOCK_FORK_X
553 JC AI_PLACED
554
555 ; --- Priority 5: Center ---
556 MOV A,CELL5
557 JNZ AI_OPP_CORNER
558 MOV CELL5,#OMARK
559 INC MOVECNT
560 LJMP AI_PLACED
561
562 AI_OPP_CORNER:
563 ; --- Priority 6: Opposite Corner ---
564 LCALL TRY_OPP_CORNER
565 JC AI_PLACED
566
567 ; --- Priority 7: Any Corner ---
568 LCALL TRY_ANY_CORNER
569 JC AI_PLACED
570
571 ; --- Priority 8: Any Side ---
572 LCALL TRY_ANY_SIDE
573 ; (C may be 0 if board full - shouldn't happen before draw detected)
574
575 AI_PLACED:

```

Line: 447	Column: 32	Insert	.asm	MCU 8051
-----------	------------	--------	------	----------

```

575 AI_PLACED:
576     RET                                ; return to GAME_LOOP; caller draws board
577
578 ;=====
579 ; TRY WIN O - find a line with 2 O + 1 empty, place O
580 ;=====
581 TRY_WIN_O:
582     MOV     TARGET,#OMARK
583     LCALL   TRY_TWO_ANY
584     RET
585
586 ;=====
587 ; TRY BLOCK X - find a line with 2 X + 1 empty, block it
588 ;=====
589 TRY_BLOCK_X:
590     MOV     TARGET,#XMARK
591     LCALL   TRY_TWO_ANY
592     RET
593
594 ;=====
595 ; TRY_FORK_O
596 ; Scan all 9 cells in order CELL1..CELL9.
597 ; For each empty cell, simulate placing O there,
598 ; count O winning threats (lines with 2 O + 1 empty).
599 ; If >= 2 threats found => it is a fork cell => place O.
600 ; Return C=1 if placed, C=0 if no fork found.
601 ;
602 ; Uses: FORK_CELL, THREAT_CNT, TARGET, R0
603 ;       (COUNT_THREATS uses ADR1,ADR2,ADR3,R5,R4,R2,DPTR)
604 ;=====
605 TRY_FORK_O:
606     MOV     FORK_CELL,#CELL1          ; start at 30H
607
608 TFO_LOOP:
609     MOV     R0,FORK_CELL
610     MOV     A,@R0
611     JNZ     TFO_NEXT                  ; cell not empty, skip
612
613     ; --- Simulate placing O ---
614     MOV     @R0,#OMARK
615
616     ; --- Count O's threats from this position ---
617     MOV     TARGET,#OMARK
618     LCALL   COUNT_THREATS

```

Line: 447 Column: 32 Insert .asm MCU 8051


```

618      ;--- Undo simulation (R0 was modified by COUNT_THREATS) ---
619      MOV     THREAT_CNT,A
620
621      ; --- Undo simulation (R0 was modified by COUNT_THREATS) ---
622      MOV     R0,FORK_CELL
623      MOV     @R0,#00H
624
625      ; --- Fork if threats >= 2 ---
626      MOV     A,THREAT_CNT
627      CJNE    A,#02H,TFO_NOT2      ; branch if A != 2
628      LJMP    TFO_FOUND            ; A == 2 => fork
629
630      TFO_NOT2:
631      JNC     TFO_FOUND            ; C=0 => A > 2 => also fork
632      SJMP    TFO_NEXT            ; C=1 => A < 2 => no fork
633
634      TFO_FOUND:
635      MOV     A,FORK_CELL
636      LCALL   PLACE_O_AT_ADDR      ; permanently place O here
637      SETB    C
638      RET
639
640      TFO_NEXT:
641      INC     FORK_CELL
642      MOV     A,FORK_CELL
643      CJNE    A,#39H,TFO_LOOP      ; 39H = CELL9+1
644      CLR     C
645      RET
646
647      ;=====
648      ; TRY_BLOCK_FORK_X
649      ; Same idea but from X's perspective:
650      ; find a cell where placing X would give X >= 2 threats,
651      ; and place O there to neutralize the fork.
652      ; Return C=1 if placed, C=0 if no X fork found.
653      ;=====
654      TRY_BLOCK_FORK_X:
655      MOV     FORK_CELL,#CELL1
656
657      TBFX_LOOP:
658      MOV     R0,FORK_CELL
659      MOV     A,@R0
660      JNZ     TBFX_NEXT
661
662      ; --- Simulate placing X ---
663      MOV     @R0,#XMARK

```

Line : 447 Column : 32 Insert .asm MCU 8051

```

664         ; --- Count X's threats ---
665         MOV     TARGET,#XMARK
666         LCALL   COUNT_THREATS
667         MOV     THREAT_CNT,A
668
669         ; --- Undo ---
670         MOV     R0,FORK_CELL
671         MOV     @R0,#00H
672
673         ; --- Block if X would get >= 2 threats ---
674         MOV     A,THREAT_CNT
675         CJNE    A,#02H,TBFX_NOT2
676         LJMP    TBFX_FOUND
677 TBFX_NOT2:
678         JNC     TBFX_FOUND
679         SJMP    TBFX_NEXT
680
681 TBFX_FOUND:
682         MOV     A,FORK_CELL
683         LCALL   PLACE_O_AT_ADDR
684         SETB    C
685         RET
686
687 TBFX_NEXT:
688         INC     FORK_CELL
689         MOV     A,FORK_CELL
690         CJNE    A,#39H,TBFX_LOOP
691         CLR     C
692         RET
693
694 ;=====
695 ; COUNT_THREATS
696 ; Input : TARGET = mark to evaluate (OMARK or XMARK)
697 ;       The test mark must already be placed on the board
698 ; Output: A = number of lines containing exactly 2 of TARGET
699 ;       and 1 empty (= immediate winning threats)
700 ;
701 ; A line is skipped entirely if it contains the opponent mark
702 ; (because that line can never be won by TARGET).
703 ;
704 ; Uses: DPTR, R5 (line counter), R4 (threat count),
705 ;       R2 (mark count per line), R0, ADR1, ADR2, ADR3

```

Line: 447 Column: 32 Insert .asm MCU 8051

```

706 ;=====
707 COUNT_THREATS:
708     MOV     DPTR,#LINE_TABLE
709     MOV     R5,#08H           ; 8 lines to test
710     MOV     R4,#00H           ; threat counter
711
712 CT_LOOP:
713     ; Load cell indices for this line and convert to RAM addresses
714     CLR     A
715     MOVC    A,@A+DPTR
716     ADD     A,#2FH
717     MOV     ADRI,A
718     INC     DPTR
719
720     CLR     A
721     MOVC    A,@A+DPTR
722     ADD     A,#2FH
723     MOV     ADRI,A
724     INC     DPTR
725
726     CLR     A
727     MOVC    A,@A+DPTR
728     ADD     A,#2FH
729     MOV     ADRI,A
730     INC     DPTR
731
732     MOV     R2,#00H           ; TARGET mark count for this line
733
734     ; --- Check cell 1 of line ---
735     MOV     R0,ADRI
736     MOV     A,@R0
737     JZ      CT_1OK           ; empty: fine, don't count
738     CJNE    A,TARGET,CT_SKIP ; opponent mark: whole line useless
739     INC     R2               ; it is TARGET mark
740 CT_1OK:
741     ; --- Check cell 2 of line ---
742     MOV     R0,ADRI
743     MOV     A,@R0
744     JZ      CT_2OK
745     CJNE    A,TARGET,CT_SKIP
746     INC     R2
747 CT_2OK:
748     ; --- Check cell 3 of line ---
749     MOV     R0,ADRI
750     MOV     A,@R0
751     JZ      CT_3OK

```

Line: 447 Column: 32 Insert .asm MCU 8051

```

751          JZ      CT_30K
752          CJNE    A,TARGET,CT_SKIP
753          INC     R2
754 CT_30K:
755          ; Exactly 2 of TARGET with no opponent => 1 empty => threat
756          CJNE    R2,#02H,CT_SKIP
757          INC     R4
758
759 CT_SKIP:
760          DJNZ    R5,CT_LOOP
761
762          MOV     A,R4                      ; return threat count
763          RET
764
765          ;=====
766          ; TRY_OPP_CORNER
767          ; If X occupies a corner, take the diagonally opposite
768          ; corner (if empty). Disrupts the "opposite corner trap".
769          ; Return C=1 if placed, C=0 otherwise.
770          ;=====
771 TRY_OPP_CORNER:
772          MOV     A,CELL1
773          CJNE    A,#XMARK,TOC_C3          ; X in CELL1?
774          MOV     A,CELL9
775          JNZ     TOC_C3                    ; CELL9 not empty, try next
776          MOV     CELL9,#OMARK
777          INC     MOVECNT
778          SETB    C
779          RET
780
781 TOC_C3:
782          MOV     A,CELL3
783          CJNE    A,#XMARK,TOC_C7
784          MOV     A,CELL7
785          JNZ     TOC_C7
786          MOV     CELL7,#OMARK
787          INC     MOVECNT
788          SETB    C
789          RET
790
791 TOC_C7:
792          MOV     A,CELL7
793          CJNE    A,#XMARK,TOC_C9
794          MOV     A,CELL3
795          JNZ     TOC_C9
796          MOV     CELL3,#OMARK

```

```

795          JNZ     TOC_C9
796          MOV     CELL3,#OMARK
797          INC     MOVECNT
798          SETB    C
799          RET
800
801 TOC_C9:
802          MOV     A,CELL9
803          CJNE    A,#XMARK,TOC_FAIL
804          MOV     A,CELL1
805          JNZ     TOC_FAIL
806          MOV     CELL1,#OMARK
807          INC     MOVECNT
808          SETB    C
809          RET
810
811 TOC_FAIL:
812          CLR     C
813          RET
814
815 ;=====
816 ; TRY_ANY_CORNER
817 ; Take any free corner (CELL1, CELL3, CELL7, CELL9).
818 ; Return C=1 if placed, C=0 if all corners occupied.
819 ;=====
820 TRY_ANY_CORNER:
821          MOV     A,CELL1
822          JNZ     TAC_3
823          MOV     CELL1,#OMARK
824          INC     MOVECNT
825          SETB    C
826          RET
827 TAC_3:
828          MOV     A,CELL3
829          JNZ     TAC_7
830          MOV     CELL3,#OMARK
831          INC     MOVECNT
832          SETB    C
833          RET
834 TAC_7:
835          MOV     A,CELL7
836          JNZ     TAC_9
837          MOV     CELL7,#OMARK

```

```

843             JNZ     TAC_FAIL
844             MOV     CELL9,#OMARK
845             INC     MOVECNT
846             SETB    C
847             RET
848 TAC_FAIL:
849             CLR     C
850             RET
851
852 ;=====
853 ; TRY_ANY_SIDE
854 ; Take any free side (CELL2, CELL4, CELL6, CELL8).
855 ; Return C=1 if placed, C=0 if all sides occupied.
856 ;=====
857 TRY_ANY_SIDE:
858             MOV     A,CELL2
859             JNZ     TAS_4
860             MOV     CELL2,#OMARK
861             INC     MOVECNT
862             SETB    C
863             RET
864 TAS_4:
865             MOV     A,CELL4
866             JNZ     TAS_6
867             MOV     CELL4,#OMARK
868             INC     MOVECNT
869             SETB    C
870             RET
871 TAS_6:
872             MOV     A,CELL6
873             JNZ     TAS_8
874             MOV     CELL6,#OMARK
875             INC     MOVECNT
876             SETB    C
877             RET
878 TAS_8:
879             MOV     A,CELL8
880             JNZ     TAS_FAIL
881             MOV     CELL8,#OMARK
882             INC     MOVECNT
883             SETB    C
884             RET
885 TAS_FAIL:
886             CLR     C
887             RET

```

```

888
889 ;=====
890 ; TRY_TWO_ANY
891 ; Scan LINE_TABLE. If a line has 2 TARGET marks + 1 empty,
892 ; place 0 in the empty cell.
893 ; Return C=1 if placed, C=0 if no such line found.
894 ;=====
895 TRY_TWO_ANY:
896     MOV     DPTR,#LINE_TABLE
897     MOV     R5,#08H
898
899 TT_LOOP:
900     CLR     A
901     MOVC    A,@A+DPTR
902     ADD     A,#2FH
903     MOV     ADRI,A
904     INC     DPTR
905
906     CLR     A
907     MOVC    A,@A+DPTR
908     ADD     A,#2FH
909     MOV     ADRI,A
910     INC     DPTR
911
912     CLR     A
913     MOVC    A,@A+DPTR
914     ADD     A,#2FH
915     MOV     ADRI,A
916     INC     DPTR
917
918     MOV     R0,ADRI
919     MOV     A,@R0
920     MOV     VAL1,A
921
922     MOV     R0,ADRI
923     MOV     A,@R0
924     MOV     VAL2,A
925
926     MOV     R0,ADRI
927     MOV     A,@R0
928     MOV     VAL3,A
929
930     ; Case 1: ADRI empty, ADRI and ADRI = TARGET
931     MOV     A,VAL1

```

```

932         JNZ     TT_CASE2
933         MOV     A,VAL2
934         CJNE    A,TARGET,TT_CASE2
935         MOV     A,VAL3
936         CJNE    A,TARGET,TT_CASE2
937         MOV     A,ADR1
938         LCALL   PLACE_O_AT_ADDR
939         RET
940
941 TT_CASE2:
942         ; Case 2: ADR2 empty, ADR1 and ADR3 = TARGET
943         MOV     A,VAL2
944         JNZ     TT_CASE3
945         MOV     A,VAL1
946         CJNE    A,TARGET,TT_CASE3
947         MOV     A,VAL3
948         CJNE    A,TARGET,TT_CASE3
949         MOV     A,ADR2
950         LCALL   PLACE_O_AT_ADDR
951         RET
952
953 TT_CASE3:
954         ; Case 3: ADR3 empty, ADR1 and ADR2 = TARGET
955         MOV     A,VAL3
956         JNZ     TT_NEXT
957         MOV     A,VAL1
958         CJNE    A,TARGET,TT_NEXT
959         MOV     A,VAL2
960         CJNE    A,TARGET,TT_NEXT
961         MOV     A,ADR3
962         LCALL   PLACE_O_AT_ADDR
963         RET
964
965 TT_NEXT:
966         DJNZ    R5,TT_LOOP
967         CLR     C
968         RET
969
970 ;=====
971 ; PLACE_O_AT_ADDR
972 ; Input:  A = cell RAM address (30H..38H)
973 ; Action: places OMARK at that address, increments MOVECNT

```



```

973 ; Action: places OMARK at that address, increments MOVECNT
974 ; Output: C=1
975 ;=====
976 PLACE_O_AT_ADDR:
977     MOV     R0,A
978     MOV     @R0,#OMARK
979     INC     MOVECNT
980     SETB    C
981     RET
982
983 ;=====
984 ; KEYPAD SCAN
985 ; Rows -> P1.0..P1.3   (output, drive low one at a time)
986 ; Cols -> P1.4..P1.7   (input,  pulled high)
987 ;
988 ; Key map:
989 ;   Row A (P1=FEH): col1=01H col2=02H col3=03H col4=0AH
990 ;   Row B (P1=FDH): col1=04H col2=05H col3=06H col4=0BH
991 ;   Row C (P1=FBH): col1=07H col2=08H col3=09H col4=0DH
992 ;   Row D (P1=F7H): col1=0CH col2=0EH col3=0FH col4=10H
993 ;=====
994 GET_KEY_BLOCK:
995 GK_WAIT:
996     LCALL   SCAN_KEY
997     JZ      GK_WAIT
998     LCALL   DELAY_DEB
999     LCALL   SCAN_KEY
1000    JZ      GK_WAIT
1001    MOV     KEYVAL,A
1002
1003 GK_REL:
1004     LCALL   SCAN_KEY
1005     JNZ     GK_REL
1006
1007     MOV     A,KEYVAL
1008     RET
1009
1010 SCAN_KEY:
1011     ; --- Row A ---
1012     MOV     P1,#0FEH
1013     NOP
1014     MOV     A,P1
1015     ANL     A,#0F0H
1016     CJNE    A,#0F0H,ROWA_HIT

```

Line: 447 Column: 32 Insert .asm MCU 8051

```

1016         CJNE    A,#0F0H,ROWA_HIT
1017         LJMP    ROW_B_SCAN
1018
1019 ROWA_HIT:
1020         CJNE    A,#0E0H,RA2
1021         MOV     A,#01H
1022         RET
1023 RA2:    CJNE    A,#0D0H,RA3
1024         MOV     A,#02H
1025         RET
1026 RA3:    CJNE    A,#0B0H,RA4
1027         MOV     A,#03H
1028         RET
1029 RA4:    CJNE    A,#070H,ROW_B_SCAN
1030         MOV     A,#0AH
1031         RET
1032
1033         ; --- Row B ---
1034 ROW_B_SCAN:
1035         MOV     P1,#0FDH
1036         NOP
1037         MOV     A,P1
1038         ANL     A,#0F0H
1039         CJNE    A,#0F0H,ROWB_HIT
1040         LJMP    ROW_C_SCAN
1041
1042 ROWB_HIT:
1043         CJNE    A,#0E0H,RB2
1044         MOV     A,#04H
1045         RET
1046 RB2:    CJNE    A,#0D0H,RB3
1047         MOV     A,#05H
1048         RET
1049 RB3:    CJNE    A,#0B0H,RB4
1050         MOV     A,#06H
1051         RET
1052 RB4:    CJNE    A,#070H,ROW_C_SCAN
1053         MOV     A,#0BH
1054         RET
1055
1056         ; --- Row C ---

```

Line: 447 Column: 32 Insert asm MCU: 8051

```

1056      ; --- Row C ---
1057 ROW_C_SCAN:
1058     MOV     P1,#0FBH
1059     NOP
1060     MOV     A,P1
1061     ANL     A,#0F0H
1062     CJNE    A,#0F0H,ROWC_HIT
1063     LJMP    ROW_D_SCAN
1064
1065 ROWC_HIT:
1066     CJNE    A,#0E0H,RC2
1067     MOV     A,#07H
1068     RET
1069 RC2:     CJNE    A,#0D0H,RC3
1070     MOV     A,#08H
1071     RET
1072 RC3:     CJNE    A,#0B0H,RC4
1073     MOV     A,#09H
1074     RET
1075 RC4:     CJNE    A,#070H,ROW_D_SCAN
1076     MOV     A,#0DH
1077     RET
1078
1079      ; --- Row D ---
1080 ROW_D_SCAN:
1081     MOV     P1,#0F7H
1082     NOP
1083     MOV     A,P1
1084     ANL     A,#0F0H
1085     CJNE    A,#0F0H,ROWD_HIT
1086     LJMP    NO_KEY
1087
1088 ROWD_HIT:
1089     CJNE    A,#0E0H,RD2
1090     MOV     A,#0CH
1091     RET

```

Line : 447	Column : 32	Insert	.asm	MCU 8051
------------	-------------	--------	------	----------

```

1092 RD2:      CJNE    A,#0D0H,RD3
1093          MOV     A,#0EH
1094          RET
1095 RD3:      CJNE    A,#0B0H,RD4
1096          MOV     A,#0FH
1097          RET
1098 RD4:      CJNE    A,#070H,NO_KEY
1099          MOV     A,#10H
1100          RET
1101
1102 NO_KEY:
1103          MOV     P1,#0FFH
1104          CLR     A
1105          RET
1106
1107 ;=====
1108 ; LCD ROUTINES
1109 ; RS -> P3.6      E -> P3.7
1110 ; D4..D7 -> P2.4..P2.7  (4-bit mode)
1111 ;=====
1112 LCD_INIT:
1113          CLR     P3.6
1114          CLR     P3.7
1115          LCALL   DELAY_BIG
1116
1117          ; 4-bit init sequence (HD44780 spec)
1118          MOV     A,#03H
1119          LCALL   LCD_WRITE_INIT_NIBBLE
1120          LCALL   DELAY_BIG
1121
1122          MOV     A,#03H
1123          LCALL   LCD_WRITE_INIT_NIBBLE
1124          LCALL   DELAY_SHORT

```

Line : 447	Column : 32	Insert	.asm	MCU 8051
------------	-------------	--------	------	----------

```

1124          LCALL    DELAY_SHORT
1125
1126          MOV      A,#03H
1127          LCALL    LCD_WRITE_INIT_NIBBLE
1128          LCALL    DELAY_SHORT
1129
1130          MOV      A,#02H                ; switch to 4-bit
1131          LCALL    LCD_WRITE_INIT_NIBBLE
1132          LCALL    DELAY_SHORT
1133
1134          MOV      A,#028H              ; 2 lines, 5x8 font
1135          LCALL    LCD_CMD
1136          MOV      A,#00CH              ; display on, cursor off
1137          LCALL    LCD_CMD
1138          MOV      A,#006H              ; entry mode
1139          LCALL    LCD_CMD
1140          MOV      A,#001H              ; clear
1141          LCALL    LCD_CMD
1142          LCALL    DELAY_BIG
1143          RET
1144
1145 LCD_CLEAR:
1146          MOV      A,#001H
1147          LCALL    LCD_CMD
1148          LCALL    DELAY_BIG
1149          RET
1150
1151 LCD_CMD:
1152          CLR      P3.6
1153          MOV      TEMP,A
1154
1155          MOV      A,TEMP
1156          SWAP     A
1157          ANL      A,#0FH
1158          LCALL    LCD_WRITE_INIT_NIBBLE
1159
1160          MOV      A,TEMP
1161          ANL      A,#0FH
1162          LCALL    LCD_WRITE_INIT_NIBBLE
1163
1164          LCALL    DELAY_SHORT
1165          RET
1166
1167 LCD_DATA:

```

Line: 447 Column: 32 Insert .asm MCU 8051

C:\Users\atiqm\Downloads\CEF | M-IDE Studio for MCS-51 : Release 0.2.5.8

```

1167 LCD_DATA:
1168     SETB     P3.6
1169     MOV      TEMP,A
1170
1171     MOV      A,TEMP
1172     SWAP     A
1173     ANL      A,#0FH
1174     LCALL    LCD_WRITE_INIT_NIBBLE
1175
1176     MOV      A,TEMP
1177     ANL      A,#0FH
1178     LCALL    LCD_WRITE_INIT_NIBBLE
1179
1180     LCALL    DELAY_SHORT
1181     RET
1182
1183 LCD_WRITE_INIT_NIBBLE:
1184     ANL      A,#0FH
1185     SWAP     A
1186     ANL      A,#0F0H
1187     MOV      R7,A
1188
1189     MOV      A,P2
1190     ANL      A,#0FH
1191     ORL      A,R7
1192     MOV      P2,A
1193
1194     SETB     P3.7
1195     LCALL    DELAY_TINY
1196     CLR      P3.7
1197     RET
1198
1199 LCD_PRINT_STRING:
1200 LPS1:
1201     CLR      A
1202     MOVC     A,@A+DPTR
1203     JZ       LPS_END
1204     LCALL    LCD_DATA
1205     INC      DPTR
1206     LJMP     LPS1
1207 LPS_END:
1208     RET
1209
1210 ;=====
1211 ; DELAYS
1212 ;=====

```

```

1212 ;=====
1213 DELAY_TINY:
1214     NOP
1215     NOP
1216     NOP
1217     NOP
1218     RET
1219
1220 DELAY_SHORT:
1221     MOV     R7,#80
1222 DS1:
1223     DJNZ    R7,DS1
1224     RET
1225
1226 DELAY_DEB:
1227     MOV     R6,#40
1228 DD1:     MOV     R7,#255
1229 DD2:     DJNZ    R7,DD2
1230         DJNZ    R6,DD1
1231     RET
1232
1233 DELAY_BIG:
1234     MOV     R6,#120
1235 DB1:     MOV     R7,#255
1236 DB2:     DJNZ    R7,DB2
1237         DJNZ    R6,DB1
1238     RET
1239
1240 ;=====
1241 ; LOOKUP TABLES
1242 ;=====
1243
1244 ; Maps cell index (0-based) to keypad label shown when empty
1245 LABEL_TABLE:
1246     DB      '7','8','9','4','5','6','1','2','3'
1247
1248 ; All 8 winning lines (stored as cell indices 1..9)
1249 LINE_TABLE:
1250     DB      1,2,3           ; row 1
1251     DB      4,5,6           ; row 2
1252     DB      7,8,9           ; row 3
1253     DB      1,4,7           ; col 1
1254     DB      2,5,8           ; col 2
1255     DB      3,6,9           ; col 3
1256     DB      1,5,9           ; diagonal TL-BR
1257     DB      3,5,7           ; diagonal TR-BL

```

Line: 447 Column: 32 Insert .asm MCU 8051

```

1258
1259 ;=====
1260 ; STRINGS (null-terminated)
1261 ;=====
1262 STR_TITLE:
1263     DB      'TIC TAC TOE',00H
1264 STR_MODE1:
1265     DB      '/=PVP  *=AI',00H
1266 STR_MODE2:
1267     DB      'ON/C = RESET',00H
1268 STR_MODE3:
1269     DB      'SELECT MODE',00H
1270
1271 STR_PVPX:
1272     DB      'PVP MODE  X TURN',00H
1273 STR_PVPO:
1274     DB      'PVP MODE  O TURN',00H
1275 STR_AIX:
1276     DB      'AI MODE   X TURN',00H
1277 STR_AIO:
1278     DB      'AI MODE   O TURN',00H
1279
1280 STR_INVALID:
1281     DB      'INVALID MOVE',00H
1282 STR_XWIN:
1283     DB      'X WINS  ON/C MENU',00H
1284 STR_OWIN:
1285     DB      'O WINS  ON/C MENU',00H
1286 STR_DRAW:
1287     DB      'DRAW    ON/C MENU',00H
1288
1289     END
1290

```

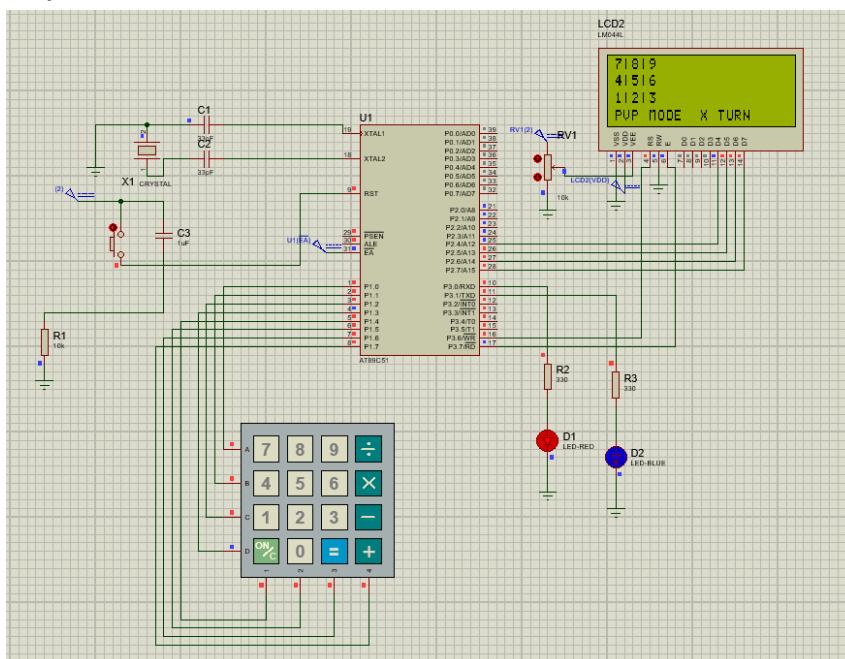
```

1258 1259 1260 1261 1262 1263 1264 1265 1266 1267 1268 1269 1270 1271 1272 1273 1274 1275 1276 1277 1278 1279 1280 1281 1282 1283 1284 1285 1286 1287 1288 1289 1290

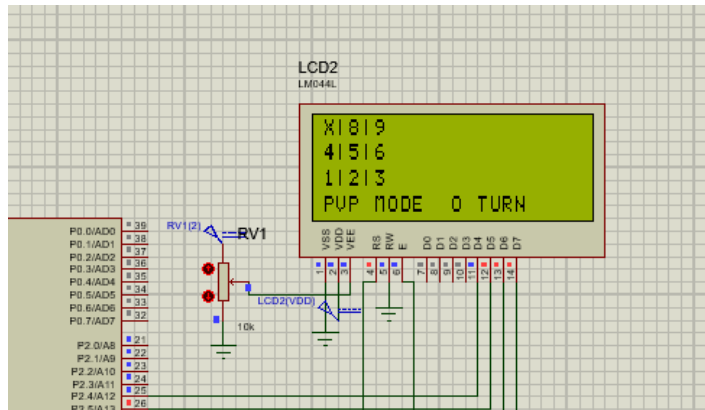
```


[illegible]

Player 2 O

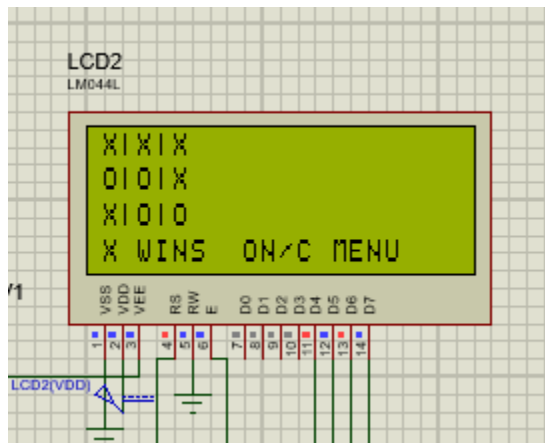


PvP Mode (Move O)

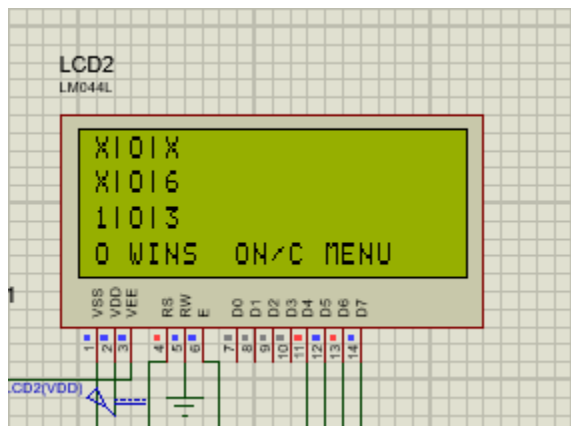


PvP Mode (Result)

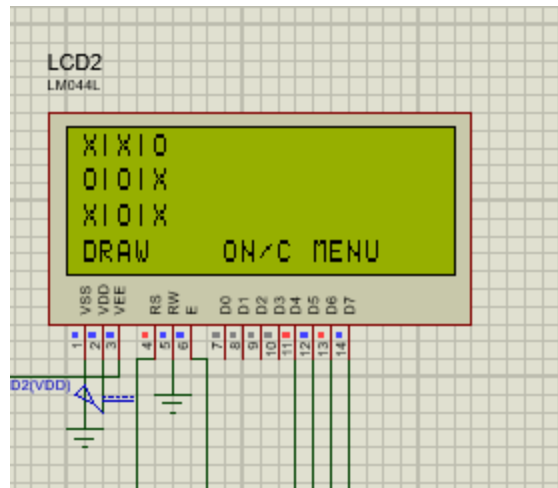
X win



O win



Draw



PvAI Mode

Player 1 X

Player 2 AI (Always O)

